Project Schedules

Gantt Chart Development & Project Scheduling — Chapter 4

Syed Samin Sadaf

Department of Computer Science and Engineering
United International University

Week 12

Lecture Roadmap

1. Building the Project Schedule
2. Allocate Resources to the Tasks
3. Identify Dependencies
4. Create the Schedule
5. Reconcile and Review the Schedule
6. Optimize the Schedule
7. Don't Abuse Buffers
8. Track the Performance of the Project
9. Managing Multiple Projects
10. Managing Commitments and Diagnosing Problems
11. Summary

Building the Project Schedule

Why the Schedule Matters

- The project schedule is **the core of the project plan**
- The project manager uses it to **commit people** to the project and show the organization how the work will be performed
- It communicates **final deadlines** and, in some cases, determines **resource needs**
- It works as a **checklist**: every necessary task must appear on it
- If a task is on the schedule, **the team is committed to doing it**

The schedule is the means by which the project manager brings the team and the project under control.

What Is a Project Schedule?

Definition

A **project schedule** is a calendar that links the tasks to be done with the resources that will do them.

Before a schedule can be created, the project manager must have:

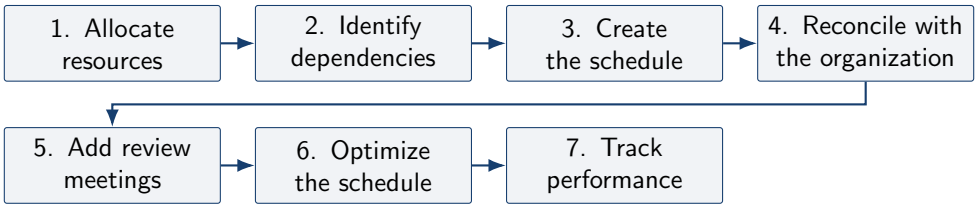
1. A **work breakdown structure (WBS)**
2. An **effort estimate** for each task
3. A **resource list** with the availability of each resource

- Without these, the result may look like a schedule but is "a work of fiction"
- A schedule is itself an estimate: dates need the **buy-in of the people doing the work**
- Build the WBS and estimates with the team first (e.g. consensus-driven **Wideband Delphi**, Chapter 3)

Concepts Before Tools

Scheduling software automates much of the tedious calculation — but the manager must first understand the concepts behind:

WBS • dependencies • resource allocation • critical paths • Gantt charts • earned value



Workflow followed in this lecture

Allocate Resources to the Tasks

What Is a Resource?

Definition

A **resource** is any person, item, tool, or service needed by the project that is either scarce or has limited availability.

"Resource" and "person" are often used interchangeably, but **people are only one kind of resource**:

- **Computer resources** — shared computer room, mainframe or server time
- **Locations** — training rooms, temporary office space
- **Services** — time from contractors, trainers, or a support team
- **Special equipment** temporarily acquired for the project

Most schedules only plan for human resources; the others are listed in the resource list of the project plan (Chapter 2).

Allocating Resources to Tasks

- **One or more resources must be allocated to each task**
- The manager identifies people on the resource list who are capable of the task and assigns it to them
- Once assigned, that team member is **not available for other tasks** until the assigned task is completed
- Some tasks can go to anyone, but **most can be done only by certain people**; if they are busy, the task must wait
- Some tasks need more than one person (e.g. a programming task may need three programmers) — then the **effort** must be divided among them

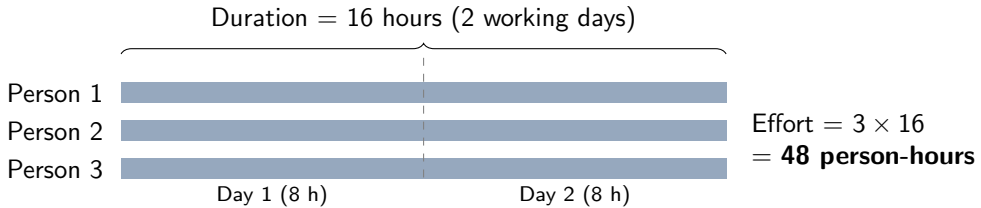
Effort vs. Duration

Duration

Time that **elapses** between starting and completing a task (hours, days, weeks). Ignores the number of people.

Effort

Total work performed, in **person-hours** (person-days, person-weeks, ...): the hours each person actually spent.



Only 5–6 of the 8 hours per day are typically spent on real engineering work. Tasks are not always divided evenly: total effort must reflect the actual time worked per resource.

Part-Time Allocation and Overhead

- A resource can be allocated to **two tasks at once** by assigning a percentage of its time to each (e.g. 50%/50%, or 30%/70%)
- Useful when a task spans several days but the resource is needed only for part of each day, or for a few days of the task
- **Overhead** = effort that does not go to the core activities of the task but is still required for people to perform it — the "real world" cost of doing the work

Example: two people instead of one

A task takes **12 days** for one person. Two people may finish in **7 days**, because they need an additional day to compare and integrate their work.

Trade-off: assigning two people requires *more total effort*, but the task has a *shorter duration*.

Compensating for Overhead and Individual Differences

- **Use the Wideband Delphi range** (an effort estimate): pick from the **low end** when fewer resources are allocated and from the **high end** for more resources; check the estimators' assumptions about staffing
- **Resources are individuals** — no two people take the same time on a task; know each member's relative expertise
- A senior programmer may take a fraction of a junior's time — but senior members are **scarce**, and some tasks are too hard for juniors
- Pairing junior with senior builds **professional development** for the next project, at the cost of training time and overhead
- Allocation has **no hard-and-fast rule**: it is often the most difficult and time-consuming part of project management, depending on skills and personal motivation

Two Principles About How People Work

Parkinson's Law (1958)

"Work expands so as to fill the time available for its completion."

Named for C. Northcote Parkinson. Extra time tends to be consumed, not saved.

Brooks (The Mythical Man-Month, 1975)

"Nine women cannot have a baby in one month."

Some tasks can be done by only one person, no matter how critical the task is — adding people does not always shorten them.

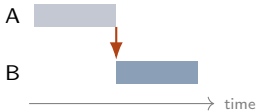
These principles return later: Parkinson's Law explains why **slack** and **buffers** do not protect a schedule.

Identify Dependencies

Task Dependencies

- Once resources are allocated, identify **dependencies** between tasks
- A task has a dependency if it involves an activity, resource, or work product that is **subsequently required by another task**
- Examples: a test plan can't run until a build is delivered; code may depend on classes built earlier; a user interface can't be built until the design is reviewed
- With Wideband Delphi, many dependencies are already captured in the estimation assumptions
- **How to record them:** number each task in the WBS and list the number of each task it depends on as its **predecessor**
- Identifying dependencies is the project manager's responsibility, working with the whole engineering team

Four Types of Predecessor



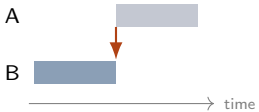
Finish-to-Start (FS)

Until the predecessor is finished, the dependent **cannot start**



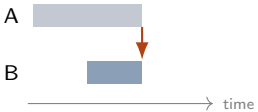
Start-to-Start (SS)

Until the predecessor starts, the dependent **cannot start**



Start-to-Finish (SF)

Until the predecessor starts, the dependent **cannot finish**



Finish-to-Finish (FF)

Until the predecessor finishes, the dependent **cannot finish**

Figure 4-1. Four different types of predecessor (A = predecessor, B = dependent; arrow = the constrained point).

Why Do Tasks Depend on Each Other?

1. Causal relationship (most common)

The dependent task relies on a work product generated by the predecessor.

Reviewers cannot review a document until it is completed.

2. Shared resource

One person cannot do two tasks simultaneously, so one task depends on the other.

If nothing forces an order, the manager and programmer may choose either.

- Project management software is the easiest way to maintain allocations and dependencies: it stores a task list with resource and predecessor information for each task
- Doing this by hand is possible, but very few project managers do

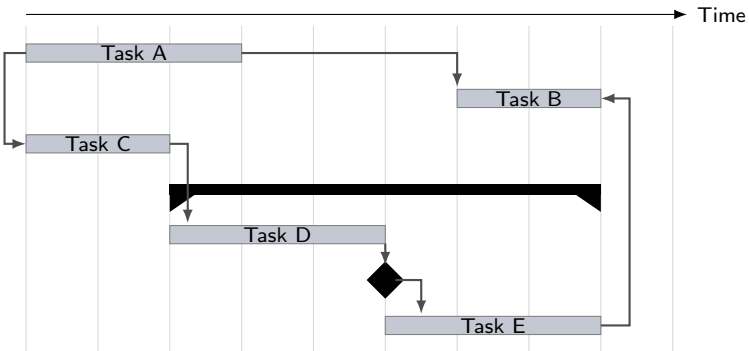
Create the Schedule

From Dependencies to a Gantt Chart

- Once resources and dependencies are set, the software arranges tasks to reflect the dependencies
- With effort and duration for each task, it calculates a **final date** and builds the schedule
- The most common form is the **Gantt chart** — a type of bar chart
- Developed by **Henry Laurence Gantt**, an American engineer prominent in the first two decades of the 20th century
- Used on major civil engineering projects (Hoover Dam, U.S. interstate highway system); now the **standard way to document software project schedules**

Reading a Gantt chart: each task is a *bar*; dependencies are *arrows* pointing to the start or the end of the task, depending on the predecessor type.

Example Gantt Chart (Figure 4-2)



- **Grey bar** = task
- **Arrow** = dependency
- **Black diamond** = milestone (a task with no duration)
- **Black bar** = summary task over D and E (its two subtasks)

Predecessor Types in Figure 4-2

Relationship	Meaning	Software example
A → B (FS)	B does not start until A is complete	Code cannot be reviewed until written: programming is FS predecessor of code review
A → C (SS)	Both start together; if A's start is delayed, C's start moves forward to match	Testers start executing tests at the same time; the build delivery is SS predecessor for the other testers' tasks
C → D (FS)	D starts after C ends; D is FS predecessor of a milestone, which is FS predecessor of E	Chain of phase-ending events
E → B (FF)	B cannot finish until E finishes; B starts later so its resources can do other work meanwhile	Test plan: FS from requirements, FF from design

Milestones, Summary Tasks, and Phases

- A **milestone** is a task with **no duration**; it marks important events in the schedule
- A **summary task** shows that its tasks are subtasks of the same parent; summary tasks can contain other summary tasks
- If an extra Wideband Delphi session decomposes a WBS task, show the original as a summary task with the new estimates as subtasks
- Schedules are normally divided into **phases** matching the major software engineering activities:

$$\text{Scope} \rightarrow \text{Requirements} \rightarrow \text{Design} \rightarrow \text{Development} \rightarrow \text{Testing}$$

- Each phase is shown as a **summary task**, with a **milestone** as its final subtask to mark the end of the phase

Reconcile and Review the Schedule

Reconcile the Schedule with the Organization's Needs

When the software's calculated due date does not fit the organization's or stakeholders' needs, work through these options in order:

1. **Reallocate resources more efficiently** — look for large gaps in the schedule; a small shift or swap of resources can close them
2. **Add or rearrange resources** (if available) — use different predecessor types: making a longer task an FS predecessor of a shorter one may open a gap that can be moved or filled with another task
3. **Find technical solutions** — revisit estimation assumptions (e.g. a command-line UI instead of a full UI; buy a component off the shelf instead of building it)
4. **Last resort: phased release** — revisit the scope, break features into phases or cut some entirely, revise the vision and scope document and re-run its review (Chapter 2)

Add Review Meetings to the Schedule

Why review?

A schedule that is put in a folder and never looked at again may as well never have been made. Nobody plans perfectly; teams improve by learning from their mistakes — so add regular review meetings to the schedule.

Progress reviews

- Held regularly to check whether the schedule is accurate and to plan action if the project goes off course
- Weekly/biweekly/monthly **status meetings** can double as progress reviews, provided the schedule details are discussed at every meeting
- The schedule is the agenda: go through each task in progress and determine its status

When a Task Is Going to Be Late

- The project manager must find a way to deal with it in the schedule
- A late task may delay other tasks — or the delay may be absorbed by the schedule
- **Stakeholders should be present**: if there are major problems they learn immediately and can help resolve them
- The usual result of the meeting is an **adjusted project schedule**
- Some delays cannot be absorbed. If an unmovable deadline will be missed, the team must either **adjust the schedule to add overtime** or return to the vision and scope document and **scale back the scope**

Progress Reviews vs. Milestone Reviews

	Progress review	**Milestone review**
When	Regularly (weekly, biweekly, monthly), often within status meetings	Scheduled in advance to coincide with project events — usually after the last task of a phase (e.g. end of design or programming)
Purpose	Track whether the schedule is accurate; plan action if the project goes off course	Review completion of a phase
Attendees	Team and stakeholders	Engineering-team representatives and stakeholders
Output	Adjusted project schedule	**Written report**: schedule delays or changes, scope modifications, and serious issues since the last milestone review — stored with the project plan

Keeping the Schedule Up to Date

- The schedule is **always updated by the project manager**, but the information used to update it should be **agreed on at status meetings**
- After every status meeting, update the schedule with the agreed changes
- A problem discovered between meetings that could cause a delay $\Rightarrow$ **notify the project manager immediately**
- If it cannot wait for the next status meeting, call an **impromptu meeting** to discuss the impact on individual tasks
- Add a **final milestone** for a postmortem meeting run by the QA lead (Chapter 8)

Optimize the Schedule

Slack

Definition

In a sequence of tasks, **slack** is the amount of time any task can be delayed without delaying the due date of the final task in the sequence.

- Most schedules contain several sequences: requirements (elicitation, documentation, verification); software (design, code, review); test plans (write, review, execute, repair, regress)
- A **tight schedule** has very little slack: any delay causes a delay in the due date
- Example: a task's owner is free only three days after its input is due $\Rightarrow$ a three-day gap. Move the task occupying that resource earlier or later, or assign another resource — this can change the final date enormously

Slack and Resource Allocation

- Apparent slack may simply mean that **all resources are allocated to other tasks**; there just happens to be time between a task and its predecessor
- Many tools summarize each resource's allocation per day — use it to find periods where resources are unallocated
- It also reveals **over-allocation**: a resource allocated more than 100% across simultaneous tasks

Over-allocation means a hidden dependency

If any resource is over-allocated, there is a dependency between two tasks that was not discovered — and the schedule is guaranteed to be inaccurate.

Slack Is Not Risk Mitigation

Common trap

Extra space between two tasks does *not* protect the second one. Parkinson's Law kicks in: the first task expands to fill the slack.

- Instead, **expand the definition of the task** to include an activity that mitigates the risk (Chapter 2), then update the schedule for the longer task
- Unavoidable slack means the schedule can tolerate some delay — but **do not rely on it**
- Better: plan other useful activities (such as **training**) for resources with slack time

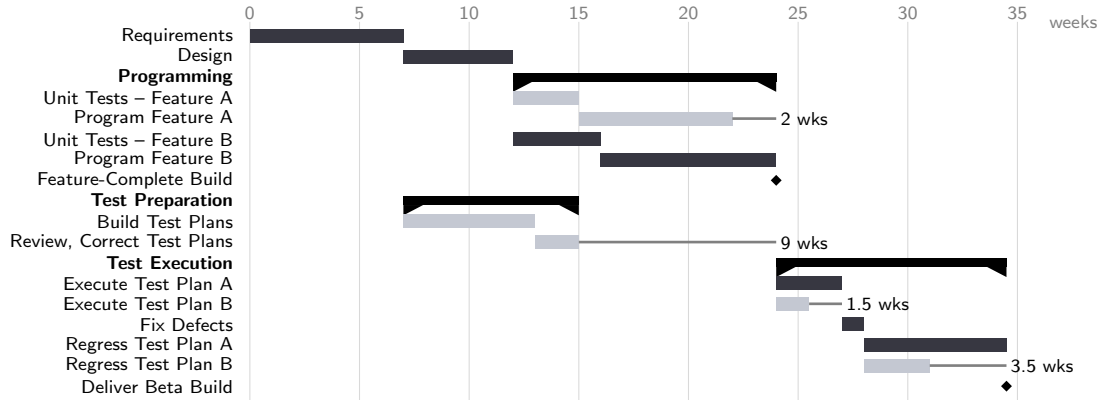
The Critical Path

Definition

The **critical path** is the sequence of tasks that represents the **minimum time required** to complete the project. If it is delayed, the schedule is delayed.

- The last task on the critical path is always the last task in the schedule
- Every schedule has **at least one** critical path; most have exactly one, but some have two or more that complete simultaneously
- **There is never slack on the critical path**
- In an optimal schedule, the critical path starts near the beginning of the project and daily effort stays relatively steady
- Monitor it closely: a late critical task delays the project; some non-critical tasks can slip without jeopardizing the due date
- Many tools highlight it on the Gantt chart

Critical Path Example (Figure 4-3)



Redrawn from Figure 4-3 (originally made with Microsoft Project 2003); positions follow the predecessor links.

Reading the Example

- Critical path (dark bars): Requirements → Design → Unit Tests B → Program B → Build → Execute Plan A → Fix Defects → Regress Plan A
- The **test-preparation tasks are not critical**: a delay in building or reviewing test plans does not move the due date — unless it exceeds **nine weeks**, which would put them on the critical path
- Thin bars to the right of tasks show slack: e.g. Execute Test Plan B would need to slip **1.5 weeks** to affect the due date

Why the critical path helps decisions

- An extra person becoming available should be assigned to **critical-path tasks**; on a non-critical task the effect on the due date is not noticeable
- It shows whether **scope creep or changed requirements** will make a difference to the delivery time

Don't Abuse Buffers

Buffers: When They Are Appropriate

Definition

A **buffer** is a task added to the schedule with no specific purpose except to account for unexpected delays — by adding extra tasks or padding existing tasks at strategic points.

Legitimate use — when the size of the loss is known

- Year-long project; every programmer takes two weeks of vacation and on average one week of sick days
- Guaranteed loss: **three person-weeks** per programmer over the year
- The manager can sprinkle three person-weeks of buffer at strategic points

Why Buffers Are Often Abused

- Expecting overruns implies **the estimate is wrong**. If so, why not increase the estimate itself?
- Buffers lull people — especially senior managers under time pressure — into thinking there is lots of "play" and that **scope can change with no impact**
- Team members feel they can ignore "small" errors in their own estimates
- If designer, programmer and tester **all** overrun and none reports it, all three count on the same buffer $\Rightarrow$ a large overrun the manager only sees after the project has slipped
- Parkinson's Law again: work expands to fill the time allocated

The Better Tool: A Risk Plan

- The project manager already has a tool for the unknown or unexpected: a **risk plan** built with the team (Chapter 2)
- Brainstorm risks and add **mitigation tasks** to the schedule — some risks are avoided, and the rest already have a plan
- Adding a risk plan means updating the schedule and re-inspecting the project plan — **this is not a bad thing**

Benefits

Updating guarantees the schedule holds no "white lies" and communicates the team's true estimate to everyone affected. Stakeholders see the team anticipated the problems and has a plan, which removes much of the pressure tied to overruns.

Track the Performance of the Project

Baseline vs. Actual Schedule

- The finished, optimized schedule is **inspected** by the project team (Chapter 2 checklist, Chapter 5); the people doing the work must agree it is realistic
- A copy of the approved version is set aside as the **baseline**: a fixed schedule used as the standard for measuring performance
- Each approved scope change (Chapter 6, change control) $\Rightarrow$ adjust the schedule and use a **new baseline revision**

Baseline version

Fixed reference; never changes over the course of the project (apart from approved scope changes).

Actual version

Updated each time a task is delayed or changed; every change stored as a separate revision (a snapshot).

Schedule Slip

Definition

When the due date of the **actual** schedule is later than that of the **baseline**, the project has **slipped**.

- The slip does not tell the whole story: a schedule may slip because the team is waiting on a single person's delayed task *or* because of a general tendency to underestimate the effort of all tasks
- Track down the **source of each slip** to improve the team's future estimates
- Work with senior management to decide whether action is needed

Variance and Earned Value

- A schedule slip does not tell the whole story — the most common way to understand it is to calculate the **variance**
- Dating from the early 1900s (factory budget vs. actual cost), variance is the core of **earned value management**: effort is "earned" against a budget only after it has actually been performed

BCWS

Budgeted cost for work scheduled: estimated effort of the tasks on the schedule to date (from the baseline).

ACWP

Actual cost of work performed: effort spent on tasks actually completed by the team (from the actual schedule).

$$\text{Variance} = \text{BCWS} - \text{ACWP}$$

Positive: fewer person-hours than budgeted. Negative: the project overran its budget.

Using Variance to Diagnose a Slip

Large slip, small variance

The variance is much smaller than the length of the delay ⇒ look for **one or two delayed tasks** (an isolated incident).

Large variance

Possible **problem with how the team estimated** tasks ⇒ spend more time on estimates, e.g. extra Wideband Delphi sessions to decompose the tasks most susceptible to delay.

This helps both **future projects** and **tasks not yet performed** in the current project.

Worked Example: Requirements Phase

Plan (baseline): 6 weeks, 2 engineers at 75% allocation on a 40-hour week

$$0.75 \times 40 = 30 \text{ h/week per engineer} \Rightarrow 2 \times 30 \times 6 = 360 \text{ person-hours (BCWS)}$$

Actual: the phase took **8 weeks** and **390 person-hours** (ACWP)

$$\text{Variance} = 360 - 390 = -30 \text{ person-hours}$$

The project **overran** its budget by 30 person-hours.

- Cost of the overrun in money = 30 person-hours $\times$ average hourly cost of engineers (plus average overhead)

Cost Performance Index (CPI)

Formula (as defined in this chapter)

$$\text{CPI} = \frac{\text{BCWS}}{\text{ACWP}} \times 100\%$$

- CPI $= 100\%$: estimate exactly right; on budget
- Requirements example: $\frac{360}{390} \times 100 = \mathbf{92.3\%}$ — below 100%: the team **did not perform as efficiently as expected** (it overran)
- CPI can compare **projects** or **phases within a project**

Note: this is the textbook's BCWS/ACWP formulation; other earned-value texts use planned value, earned value and actual cost.

Interpreting CPI

- **Pinpoint problem phases.** If programming took twice as long as estimated but all other phases came in under estimate, total variance may look small — CPI **per phase** reveals the problem, and the manager should give extra attention to programming estimates in future projects
- **Track estimation skill.** Compare overall CPI across projects; CPI moving toward 100% shows estimation is improving. Chart CPI for each completed phase; many phases far from 100% $\Rightarrow$ feed this into future estimation sessions
- **Find systemic process problems.** Consistently poor CPI in coding across many projects may mean flawed programming estimates, problems executing coding tasks, or uncontrolled changes that go unnoticed until coding begins
- **Isolated or systematic?** If other projects' requirements-phase CPI averages about 90%, the team systematically underestimates requirements; if it is close to 100%, a specific problem hit this project

Tracking Progress at Review Meetings

- Track progress in terms of **slips, variance, and earned value**
- Simplest method: compare the actual schedule's due date with the baseline's due date to anticipate the expected delay
- Record variance data, the individual delays behind it, and any theories about why the schedule slipped — used in the **postmortem report** (Chapter 8)

Tools: Microsoft Project is the most popular scheduling tool. Free/open-source options with task lists, resource allocation, predecessors, and Gantt charts include **Open Workbench**, **dotProject**, **netOffice**, and **TUTOS**.

Managing Multiple Projects

Managing Multiple Projects

- Many project managers are responsible for several projects
- If each project is well planned, managing the set is not difficult
- **No shared dependencies:** manage each project individually, with a separate schedule
- **Shared dependencies** are harder. There are two types:

1. Shared resources

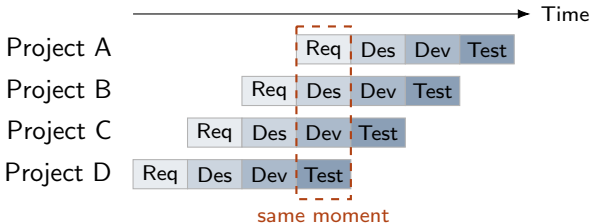
Two projects rely on the same resources.

2. Shared work products

A work product generated by one project is needed by another.

Getting a handle on these dependencies is the first step.

Pipelined Projects



Each phase uses only a subset of the team, so the rest can work on other projects — at the same moment A is in requirements, B in design, C in development, and D in testing.

- **Trouble:** no two projects take exactly the same time, and phases need different shares of effort (programming is typically 30–40% of the total). A lead tester may be on a test plan while programmers stop for half a day to review another project's requirements or design
- With more than two or three simultaneous projects, things can get **very chaotic**

Over-Allocation and Under-Allocation

Allocation can be **full** (a programmer on a task) or **partial** (a room reserved only mornings; a designer splitting the day between two projects).

Over-allocation

- First goal: shared resources must not be over-allocated
- 50% on one project + 100% on another in the same week = **150%**; this often does not show on either schedule
- Estimates that omit overhead (meetings, email, interruptions): 8 office hours may be only **5 hours** of project work
- Scope changes without re-estimation over-allocate people

Under-allocation

- An engineer with no tasks may get bored; teammates who are crunched see them leave early — **morale suffers**
- Remedy: low-priority projects whose tasks go to under-allocated people
- Allow enough time to ramp up and finish, or it feels like **busywork**

Tools and Cross-Project Dependencies

- **Resource pool** feature: one set of resources shared by many projects; drawing a resource in one schedule raises its allocation in the pool and therefore in *all* the other schedules
- Run **allocation reports** to verify no resource is over- or under-allocated; alternatively put multiple projects on a single schedule (harder to maintain, easy to see at a glance)
- **Cross-project predecessors:** whenever a task relies on software, a document, or a work product scheduled in another project, a dependency exists. Easiest handling: the dependent task **does not begin until its predecessor ends**; tools can group projects into one **master schedule**

Pitfall

Each cross-project dependency increases the risk of delay on the dependent project. Mitigation: review **every predecessor** at the dependent project's meetings.

Prioritize Projects Realistically

- Like prioritizing tasks — tough decisions that leave someone unhappy. Priorities are relative: each stakeholder thinks their project is most important
- It is more **politically charged**, so managers are tempted to pretend all projects can be done at once. **This is usually a mistake**: unless projects share no resources, some resource will eventually have to choose one or the other
- Someone must "put his foot down" that project A is more important than project B — sometimes the project manager, more often senior managers (e.g. a **steering committee**)
- **Follow the money:** whoever allocates funds for engineers and computers usually decides which project comes first; ideally a single person (often the committee chair)
- If no decision-maker can be found, finding one becomes the project manager's **most important responsibility**

The Prioritization Process

1. Rank like risks, but **no two projects share a priority**
2. With 20 projects: pick the top one → priority 1; the lowest → priority 20; place each other project relative to those
3. If undecidable, flip a coin — if the result feels wrong, reverse it (this forces consideration of the alternative)
4. Result: an ordered list with a **unique priority** per project, also showing which projects depend on others

Why unique priorities?

Otherwise the team ends up with several "top priority" projects and no idea which to work on next; if many projects have priority 1, prioritization is essentially useless.

Using Priorities Day to Day

- Assign each resource to the next task on the **highest-priority project** that does not yet have work being done
- Not a hard-and-fast rule: a task on a lower-priority project may be needed to make schedules work or avoid over-allocation, and often only one person has the expertise — but wherever possible, **honor the priorities**
- Hold a **periodic priority meeting** (weekly, biweekly, or monthly) to reevaluate priorities; it can be piggybacked on the status meeting if all decision-makers attend
- Everyone stays in the loop: the project manager, the lead decision-maker, and the stakeholders

Managing Commitments and Diagnosing Problems

Use the Schedule to Manage Commitments

- A schedule is a **commitment** by the team: once a task is agreed, its assignee is committed to finish by the due date
- Senior managers treat the schedule as an accurate forecast — a slip is an exception needing explanation
- **Problem:** changes announced in a meeting (e.g. "start doing code reviews") often fail to stick; a disagreeing member stays silent and builds as before, claiming there is no time
- **Solution:** add tasks for the improvement to the schedule (schedule all the review meetings)

Benefits of scheduling the change

Time is built in; adoption is clearly expected; and it is a **consensus-building tool** — disagreement surfaces at the schedule review, so the change is discussed openly or gains real commitment.

Scheduling Problem: Working Backward from a Deadline

- A deadline that seemed reasonable at the start becomes unrealistic as it nears — often because it came from **marketing or customer relations**, not from estimates of actual effort
- **Misguided approach:** split the project into phases with fixed percentages (e.g. programming 60%, testing 25%). These numbers come from the need to fit a predetermined time, not from knowledge of the work
- **Result: deadline-driven work** — reviews, inspections, even testing are abandoned; people are told to work overtime or weekends
- Unfair to stakeholders too: the client is blamed for expecting too much, or marketing for over-promising

Whose Fault Is an Unrealistic Schedule?

The project manager's

The manager agreed to an unrealistic schedule instead of being honest about the likelihood of failure and presenting alternatives.

Alternatives to offer stakeholders up front:

- Add resources
- Reduce scope
- Propose a phased release
- Bring on consultants
- Use different technologies

Had the manager been upfront from the beginning, the stakeholders might have avoided the mess.

Scheduling Problem: Misunderstood Predecessors

- A reasonable deadline can still fail if dependencies were not understood; one discovered halfway through can send the team into chaos
- Most common when **no WBS was created**: important tasks appear well after work has started
- Example: a programmer discovers he needs a component another member won't build for two weeks; the code he is building is needed by yet another programmer. Some members work overtime while others sit idle
- Few opportunities for correction: key people are on other tasks and end dates are already agreed

Cascading Delays

- In a tight schedule, when one task has to wait, everything that depends on it waits, and so on — delays **cascade**
- Often not recognized until late: each person finishes their task in the estimated time, so progress looks steady until the project nears completion and the deadline is in jeopardy

Who suffers most?

Testers — they own the tail-end tasks, so they carry the bulk of the remaining work and bear the brunt of the pressure even though they did not create the problem.

Summary

Key Takeaways

1. A schedule links **tasks** to **resources** over a calendar; it needs a WBS, effort estimates, and a resource list first
2. Distinguish **effort** (person-hours) from **duration** (elapsed time); account for **overhead**
3. Identify **dependencies** (FS, SS, SF, FF) — causal or shared-resource
4. The **Gantt chart** shows bars, dependency arrows, milestones, and summary tasks
5. Reconcile with deadlines by reallocating resources, using technical solutions, or (last resort) phasing the release
6. Add **progress and milestone reviews**; keep the schedule current
7. **Slack** is not risk protection; monitor the **critical path**; use **buffers** only for known losses
8. Track against a **baseline**: slip, **variance**, and **CPI**
9. With multiple projects: avoid over-/under-allocation and give each project a **unique priority**
10. The schedule is a **commitment** tool; deadline-driven and predecessor-blind scheduling are the classic failures

Review Questions

1. What must exist before a project schedule can be built, and why does a schedule need the team's buy-in?
2. Three people work together on a task for 2 working days (8 h/day). What are the duration and the effort?
3. Give one example each of an FS, SS, and FF predecessor from a software project.
4. Why is using slack as risk protection usually a mistake? What should be done instead?
5. A phase was budgeted at 240 person-hours and took 300. Compute the variance and the CPI. *(Answer: -60 person-hours; 80%.)*
6. Why should an extra team member be assigned to critical-path tasks?
7. Why must every project have a unique priority?